

Front End Developer Exam

Introduction

This exam will use a Sound API to fetch data and display it on a page. The Sound API is a platform that gives users access to a library of tracks. These tracks can be embedded and played on other web pages. The Sound API allows you to query the tracks.

Apart from the Sound API, you are allowed to make use of any library you see fit to help you complete the task. Assume you are developing this for the latest Chrome release, so no need to support old browsers.

Once you have completed the project, please upload your code to a Git repository (e.g., GitHub, GitLab) and share the repository link. Ensure that the repository is public or provide access permissions if it is private.

Attach a readme file with instructions on how to run the project.

Sound API Options

1. Mixcloud - <https://www.mixcloud.com/developers/>

- API Query - <https://api.mixcloud.com/search/?q=adele&type=cloudcast>

- Embedding -

https://www.mixcloud.com/widget/iframe/feed=https://www.mixcloud.com/shinshinobie/adele/&hide_cover=1&light=1

2. Any other sound API

The Layout



Create an HTML Page with the Following Items:

1. Search Container which contains:

- A text box
- A button or link to trigger the search
- A container for the search results
- Pagination & view controls at the bottom: **Previous**, **Next**, **List** and **Tile** buttons

2. Image Container

3. Recent Searches Container

The Task

MANDATORY required **BONUS** nice to have

- MANDATORY** 1. Use the Sound API to allow the user to search for anything entered in the search box. The names of the results should be displayed as a list below the textbox. Only fetch 6 results at a time.
- MANDATORY** 2. When clicked, the Next button should display the next 6 results instead of the current results. Add a matching **Previous** button. **Next must be disabled when there are no further results** (use the API's paging cursor rather than a naive offset).
- MANDATORY** 3. Keep a history of the searches for the user in the Recent Searches Container. This history should display the last 5 searches and should be available for the user in subsequent visits. **Duplicate terms must not create duplicate entries** — re-searching a term moves it to the top.
- MANDATORY** 4. When clicked, a search item should instigate a new search.
- MANDATORY** 5. When a search result is clicked, it should fly to the Image Container, fade out, and the image for the search result should fade in instead.
- MANDATORY** 6. When clicking the central image, embed the track below the image and make it play.
- MANDATORY** 7. **Correct async handling.** Debounce the search input (~300ms) and cancel in-flight requests (`AbortController`): if the user starts a new search before the previous one resolves, the stale response must *never* overwrite the current results. Rapidly clicking Next/Previous while a request is pending must not corrupt the displayed page.
- MANDATORY** 8. **Handle every state gracefully:** a loading indicator while fetching, an empty state when there are no matches, and a visible error with a retry option on network/API failure. No silent dead pages.
- MANDATORY** 9. **TypeScript with properly typed data** (no any), and the core logic (history dedup, pagination cursor, storage layer) kept separate from the view — with a few **unit tests** covering it.
- MANDATORY** 10. **Smart architecture.** Structure the app into clear, decoupled layers — an API/data layer, reusable UI components, and state/business logic — so that the data source, the view, and the state each change independently. We should be able to swap the Sound API for another provider by touching only the data layer, and reuse a component without dragging its data-fetching along. Explain the structure and the reasoning behind it in the README.
- BONUS** 11. When clicking the Tile button, the search results should tile in rows using their images. Clicking the List button should toggle back to list view. This selection should be remembered for the user's next visit.
- BONUS** 12. **Accessibility:** the full flow is keyboard-navigable, HTML is semantic, and ARIA is used where needed (e.g. the results list, and focus management when a result flies to the Image Container).
- BONUS** 13. Implement a beautiful design with well-structured CSS to enhance the user experience.

What We're Looking For

- Well-written code broken into logical components that interact with each other.
- Smart, decoupled architecture: distinct data, UI, and state layers; the data source is swappable without touching the view.
- Logic separated from the view layer; components composed and reusable.
- React (with TypeScript) is strongly preferred. **Avoid a one-page jQuery solution.**
- Correct async handling: debounce, in-flight cancellation, no stale-response races.
- Graceful loading, empty, and error states.
- Accessibility: keyboard-navigable, semantic HTML, ARIA where needed.
- At least a few unit tests on the core logic.

- A README explaining architecture choices and trade-offs.
- Clean, incremental git history — not a single "final commit" dump.

Contact Information

For any consultations, questions, or further assistance, please contact:

Shmuel Elkis

Mobile Development Team Leader

Priority Retail

+972-52-578-5401

shmuel.elkis@priority-software.com